ECAI 2025
I. Lynce et al. (Eds.)

© 2025 The Authors.
This article is published online with Open Access by IOS Press and distributed under the terms of the Creative Commons Attribution Non-Commercial License 4.0 (CC BY-NC 4.0).
doi:10.3233/FAIA251403

Search Trajectory Network-Enhanced Multi-Objective Dynamic Algorithm Configuration

Robbert Reijnen[a,*], Zaharah Bukhsh[a], Hoong Chuin Lau[b], Yaoxin Wu[a] and Yingqian Zhang[a]

[a]Eindhoven University of Technology
[b]Singapore Management University

Abstract. Deep reinforcement learning (DRL) has emerged as an effective technique for dynamic algorithm configuration, particularly in evolutionary computation, enabling adaptive parameter updates during algorithmic execution. DRL-based methods have shown broad applicability across different problem domains and are designed to configure algorithms without problem-specific information, making them highly transferable across problem variants and scalable to different problem sizes. This paper proposes a novel graph neural network-based approach that learns representations of Search Trajectory Networks (STNs) to track the convergence behavior of multiple objectives and dynamically reconfigures multi-objective evolutionary algorithms during execution. By capturing how solutions evolve and interact over time, the STN-based state representation enables real-time insight into convergence, diversity, and their trade-offs, facilitating more informed and adaptive configuration decisions. Extensive experiments indicate that our method outperforms the state-of-the-art DRL-based algorithm configuration methods. It also demonstrates good scalability to large problem instances and effectiveness in real-world optimization problems, which are often computationally expensive to tune.

1 Introduction

The performance of optimization algorithms is highly dependent on the selection of appropriate parameters. Traditional approaches to algorithm configuration often rely on manual tuning, which is time consuming and requires expert knowledge. To address this, automated algorithm configuration (AC) methods have been developed to systematically explore parameter spaces and identify well-performing configurations [13, 14]. However, these methods typically yield static configurations that may not adapt well to changes in the optimization landscape. Dynamic Algorithm Configuration (DAC) has emerged as a promising alternative, allowing the adaptation of parameters in real time during the execution of an algorithm [3]. This is particularly beneficial for iterative methods such as Evolutionary Algorithms (EAs), which benefit from different parameter settings at various stages of the optimization process.

Recently, Reinforcement Learning (RL) has gained considerable attention for dynamic algorithm configuration, particularly in the context of EAs [18, 6]. In this domain, parameter configurations are treated as actions, and a reward is given to the RL agent when a configuration set leads to improved solutions, encouraging the RL agent to choose desirable parameters. Deep Reinforcement Learning (DRL) has further advanced this, replacing tabular representations of RL agents with deep neural networks to model complex decision-making processes (e.g. [25, 27, 21, 2]. DRL has shown promising results in dynamic algorithm configuration for multi-objective optimization, with studies exploring its potential to handle the trade-offs between conflicting objectives [9, 30, 19].

Despite these advancements, the application of DRL to multi-objective optimization still presents several challenges. Many of the existing approaches define the states in the MDP using manually configured features derived from convergence and fitness landscapes, such as the number of elite solutions, solution spacing, the relative size of gaps in the solution space, and hypervolume. This process is labor-intensive and often suboptimal, as it requires expert knowledge to define features that appropriately capture the problem's dynamics. Moreover, high-dimensional state spaces and the optimization of multiple objectives further complicate the learning process [33]. Furthermore, many of the current studies in this field focus on search operator selection, typically formulated as discretized actions, and are often demonstrated on simple continuous optimization problems or standard benchmark functions [15]. This limits the generalizability and applicability of the findings to more complex and diverse problem domains, such as those found in combinatorial optimization.

To address these challenges, we propose a DRL-based approach called *Search Trajectory Network-based Multi-Objective Dynamic Algorithm Configuration (STN-MODAC)*. STN-MODAC leverages a Search Trajectory Network (STN) to perform dynamic algorithm configuration for solving multi-objective combinatorial optimization (MOCO) problems. The STN represents the optimization process as a graph in the objective space, where the nodes correspond to the individual solutions encountered throughout the search, and the edges capture the transitions between the solutions over time. As such, the STN encodes not only the solutions currently present in the population but also the search progress towards it, providing an indication on the algorithmic performance. Using STN, our method dynamically learns representations of the optimization state across multiple objectives, adapting as the search evolves. Importantly, the STN provides a graph-based representation that is independent of the number of objectives, removing the need for manual design of hand-crafted state features and enabling generalization across problem types. To process the STN, we embed it using a Temporal Graph Convolutional Network (T-GCN), which captures both the structural and temporal dynamics of the search process. This embedding serves as the input state to the policy network, allowing the agent to learn from evolving patterns within the optimization trajectory.

* Corresponding Author. Email: r.v.j.reijnen@tue.nl.

Experiments show that the proposed methods outperform widely adopted algorithm configuration methods, such as irace [14], SMAC3 [13], and state-of-the-art multi-objective DRL approaches such as MADAC [32] and GS-MODAC [22]. Furthermore, the trained models show strong generalization capabilities, successfully solving larger instances and more constrained variants of the problem, derived from real-world problems. Specifically, we investigate the use of cheaper-to-compute problem variants for training, with the goal of transferring the learned policies to more challenging, real-world problem instances. This approach allows for the development of effective policies without the need to train directly on computationally expensive problem variants, thereby reducing the computational burden while still yielding effective and transferable policies.

Our main contributions are as follows:

- We propose STN-MODAC, a graph-based DRL framework for dynamic algorithm configuration in MOCO, which models the search process using a Search Trajectory Network (STN). This approach captures the dynamic behavior of multi-objective optimization by representing convergence and diversity over time in a structured, objective-space graph, which eliminates the need for handcrafted features, and provides a scalable state representation that is agnostic to the number of objectives.
- We empirically demonstrate that STN-MODAC consistently outperforms established algorithm configuration approaches (e.g., irace, SMAC3), as well as advanced DRL-based methods for dynamic configuration in multi-objective optimization. We further demonstrate that the proposed model, trained on simple problem variants, generalizes favorably to larger problem instances and complex real-world problem variants with additional constraints, reducing training overhead while preserving performance.

2 Preliminaries and Related Work

Multi-Objective Combinatorial Optimization (MOCO) Combinatorial optimization focuses on finding the best solution from a finite set of feasible solutions. These problems are characterized by their discrete nature, where the solutions can be represented as integers, graphs, sets, or sequences. Multi-Objective Combinatorial Optimization (MOCO) involves simultaneously optimizing multiple, often conflicting objectives for combinatorial optimization problems. The general formulation of MOCO can be expressed as $\min_{x \in X} f(x) = (f_1(x), f_2(x), \ldots, f_N(x))$. Here, X denotes the set of feasible solutions, N is the number of objective functions to be optimized, and each $f_i(x)$ represents an objective to be minimized.

Definition 1: Pareto Dominance. A solution $x_1 \in X$ dominates another solution $x_2 \in X$ ($x_1 \prec x_2$) if and only if: $f_i(x_1) \leq f_i(x_2)$ for all $i \in \{1, \ldots, N\}$, and there exists at least one $j \in \{1, \ldots, N\}$ such that $f_j(x_1) < f_j(x_2)$.

Definition 2: Pareto Optimality. A solution $x^* \in X$ is considered Pareto optimal if there is no other solution $x' \in X$ satisfying $x' \prec x^*$. In other words, x^* is Pareto optimal if it is not dominated by any other solution in X.

Definition 3: Pareto Front. The objective of multi-objective optimization is to find the Pareto front, which consists of all Pareto-optimal solutions: $\mathcal{P} = \{x^* \in X \mid \nexists x' \in X \text{ such that } x^* \prec x'\}$. The corresponding Pareto front is defined as: $\mathcal{F} = \{f(x) \mid x \in \mathcal{P}\}$. The Pareto front consists of the objective values of the Pareto set, where each $f(x)$ represents a point in the objective space.

Definition 4: Hypervolume Indicator. The Hypervolume (HV) indicator is a widely used metric to assess performance in multi-objective optimization problems, assessing both convergence and diversity without requiring knowledge of the exact Pareto front [34]. For a Pareto front $\mathcal{F}$ in the objective space, the HV with respect to a fixed reference point $r \in \mathcal{R}^N$ is defined as: $\text{HV}_r(\mathcal{F}) = \mu\big(\bigcup_{f(x) \in \mathcal{F}} [f(x), r]\big)$, where μ is the Lebesgue measure, representing the N-dimensional volume, and $[f(x), r]$ refers to an N-dimensional cube: $[f(x), r] = [f_1(x), r_1] \times \cdots \times [f_N(x), r_N]$, spanning the region in the objective space between a point on Pareto front and a fixed reference point r.

Dynamic Algorithm Configuration (DAC). Algorithm Configuration involves selecting optimal parameter settings to maximize an algorithm performance on different problem instances. Traditional AC methods typically produce static configurations that, while effective in some cases, may not adapt well to dynamic problem landscapes. To address this limitation, Dynamic Algorithm Configuration (DAC) has been introduced, allowing parameter adjustments during algorithm execution to improve performance [3]. Unlike static configurations, DAC facilitates a balance between exploration and exploitation, improving the adaptability of optimization methods. DAC can be categorized into three main approaches [10]: 1) *Deterministic Methods*, modifying parameters based on predefined schedules, such as time-dependent functions [26], 2) *Self-Adaptive Methods*, with parameters evolving along with solutions, integrating their adjustments directly into the search process [16], and 3) *Adaptive Parameter Control*, in which parameters are modified in response to search dynamics, using credit assignment and operator selection techniques to enhance performance [1].

Reinforcement Learning for DAC More recently, Reinforcement Learning (RL) has gained prominence in DAC, particularly in Evolutionary Algorithms (EAs). RL approaches model parameter selection as a sequential decision-making problem, where optimal configurations are learned through interaction with the optimization process. When a parameter setting leads to improved solutions, a reward signal reinforces beneficial adjustments. This can be formulated as a contextual Markov Decision Process (MDP) M_I, with shared action and state spaces, but unique transition and reward functions for each instance i [3]. Each M_i represents an MDP specific to the problem instance $i \in I$, with shared state S, and action space A, and instance-specific transition T_i, and reward function R_i.

The state space S provides the DRL agent with information on the current status of the optimization process, helping to select the best action for the next iteration. Various studies have explored state representations that effectively capture the convergence process and generalize to new problem instances [25, 26, 32]. These typically include convergence indicators, objective values, and diversity metrics. The transition T_i models the dynamics of the optimization process, determined by the interactions between the agent and the problem environment. Each interaction corresponds to an optimization step, where, given a state s_t, the agent takes an action a_t, and the probability of transitioning to the next state s_{t+1} is given by $T(s_{t+1}|s_t, a_t)$. Unlike the state and action spaces, the reward and transition functions are specific to each instance $i \in I$. The action space A consists of continuous or discrete values corresponding to the algorithmic parameters to be controlled. These values are normalized to a predefined range to ensure stability and consistency across different configurations, and the reward function R encourages convergence towards optimal solutions.

Several studies have demonstrated the effectiveness of RL. For example, Q-learning has been applied to dynamically adjust crossover

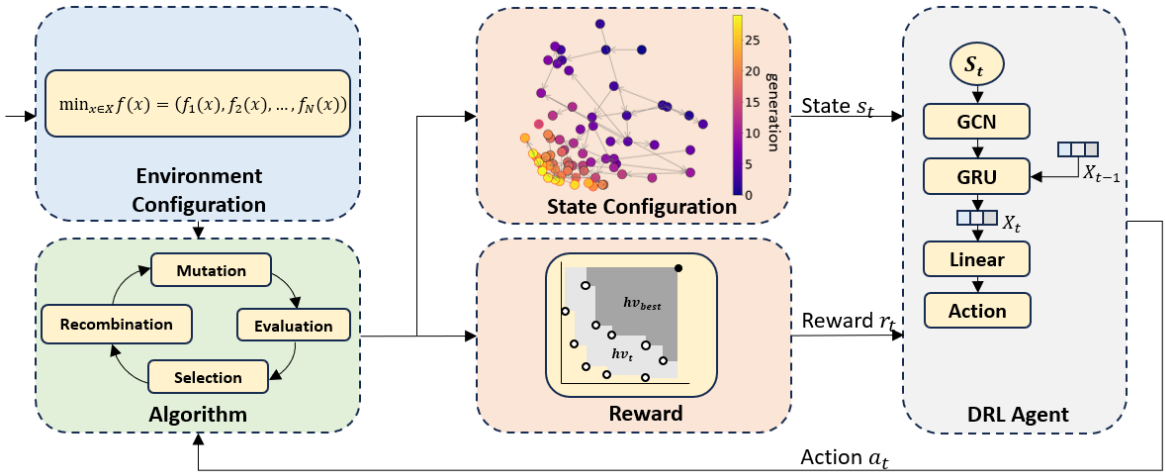


Figure 1: The STN-MODAC framework. The DRL agent configures the next iteration of the search algorithm based on a Search Trajectory Network (STN), where nodes represent non-dominated solutions from the Pareto front of different generations, and edges capture parent-child relationships between solutions. Node features include objective values and a flag indicating if a solution is present in the current population. A Temporal Graph Convolutional Network (T-GCN), combining GCNs with a GRU, processes the STN by integrating embeddings from prior states to capture structural and temporal dynamics (i.e. X_{t-1}). The agent uses the processed representation to predict actions for configuring the algorithm, and the environment returns the resulting next state and reward.

and mutation rates in vehicle routing problems [18], while SARSA and Q-learning have been used to optimize parameter control in job shop scheduling [6]. Building on these approaches, Deep Reinforcement Learning (DRL) has shown promise in DAC by using neural networks to approximate optimal configuration policies. Notable applications include the use of deep Q-networks (DQN) for parameter selection in Differential Evolution [25] and policy gradient methods for adaptive control in EAs [27]. Despite these advancements, applying DRL to multi-objective optimization presents unique challenges. Many existing approaches rely on manually engineered features, such as convergence metrics and fitness landscape properties, to define MDP states. This manual feature selection can be labor intensive and suboptimal. Furthermore, managing high-dimensional state spaces and optimizing across multiple competing objectives increases the complexity of learning effective policies [33]. Furthermore, most previous work has focused on discrete action spaces for the selection of search operators, with limited application to real-world combinatorial optimization problems [15].

3 Method

In this section, we introduce our approach *Search Trajectory Network-based Multi-Objective Dynamic Algorithm Configuration (STN-MODAC)*. Our method harnesses the representational power of Search Trajectory Networks (STNs) to model both the spatial distribution and the temporal evolution of solutions for Multi-Objective Combinatorial Optimization (MOCO) tasks. By encoding rich structural and temporal information from the search trajectory, STN-MODAC dynamically tunes the parameters of Evolutionary Algorithms (EAs) to promote both convergence and solution diversity across different optimization landscapes. In the following sections, we will introduce STNs, and then elaborate on both the MDP formulation of the proposed method and its GNN-based training algorithm. We illustrate the overview of STN-MODAC in Figure 1.

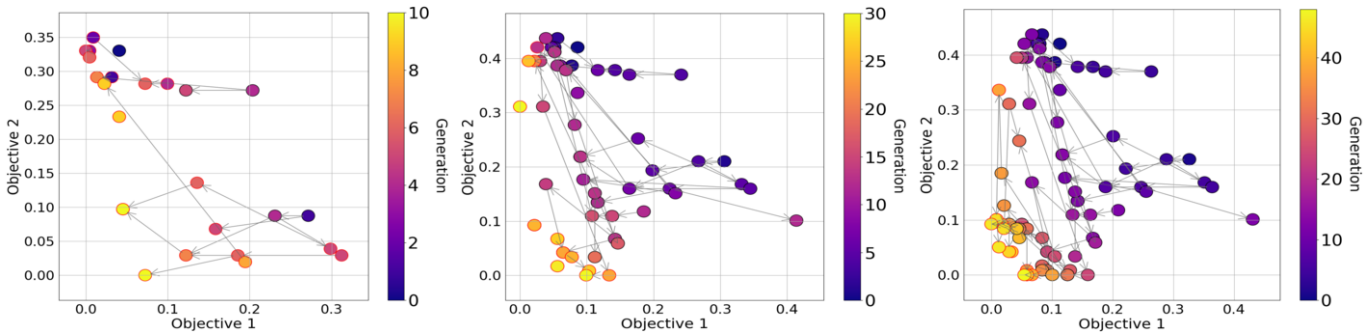
3.1 Search Trajectory Networks (STN)

This work examines the use of Search Trajectory Networks (STNs) to provide convergence information for deep reinforcement learning algorithms. Search Trajectory Networks (STNs) are a structured method to analyze and visualize the behavior of optimization algorithms by capturing how solutions evolve over time within the search space [17]. STNs represent the trajectory of a search process by organizing solutions encountered during the search into a graph structure. Analyzing the network of nodes and edges can reveal areas of frequent exploration and provides insight into convergence behavior.

An STN is formally defined as a directed graph $STN = G(N, E)$, where N is the set of nodes that correspond to solutions in various time steps, and E represents the set of directed edges between these nodes. The edge weight reflects the transitions between solutions, allowing visualization of the algorithmic convergence through the search space and its convergence trajectory. This provides a powerful alternative to traditional diversity metrics and visualization techniques, such as Pareto front approximations and trace generation plots, to analyze convergence [5, 31, 8].

In multi-objective optimization, where solutions are represented as vectors of trade-offs between multiple conflicting objectives, the STN framework is extended to Multi-Objective Search Trajectory Networks (MO-STNs) [12]. MO-STNs track the evolution of solutions across multiple objectives, with each node in the network representing a multidimensional solution vector. Unlike single-objective STNs, which track a single solution per iteration, MO-STNs monitor a set of representative solutions that capture different trade-offs in the objective space. These solutions are selected using a set of decomposition vectors that partition the objective space into subregions aligned with various preferences among the objectives.

At each iteration, candidate solutions are identified via non-dominated sorting, and representative solutions are selected. This ensures that each vector is associated with a solution that best reflects the trade-off between the objectives. The selected solutions are then mapped to nodes in the MO-STN, where each node corresponds to a set of objective values in the objective space. Transitions between solutions across iterations are captured as directed edges to reflect the transitions, providing a dynamic and spatially structured view of the search behavior in the multiobjective setting.



(a) Generation 10 (b) Generation 30 (c) Generation 50

Figure 2: Visualization of the evolving Multi-Objective Search Trajectory Network (MO-STN) at generations (a) 10, (b) 30, and (c) 50. Each MO-STN captures the dynamic structure of the optimization process by encoding non-dominated solutions (nodes), their ancestry (directed edges), and temporal information (edge generation index). Node features include objective values and generation membership, while edges represent parent-offspring relationships. These visualizations illustrate how the search space structure develops over time, preserving influential solutions and highlighting the state transitions in STN-MODAC method.

3.2 Markov Decision Process Formulation

The optimization process is framed as a contextual Markov Decision Process (MDP), denoted M_I. Here, all problem instances $i \in I$ share a common action and state space, while differing in their specific transition and reward functions, following the paradigm introduced by Biedenkapp et al. [3]. We formulate the MDP as follows:

State Space S — State is represented by a *Multi-Objective Search Trajectory Network (MO-STN)*, a dynamic directed graph designed to capture the history and structure of the optimization process. The MO-STN reflects both the ancestry of solutions and their temporal development over generations. It is composed as follows:

- **Nodes:** Correspond to non-dominated solutions that have appeared on the Pareto front at any generation. Each included node persists in the network over generations, enabling the tracking of influential solutions that have appeared in earlier identified fronts.
- **Node Features:** Include the objective values (that is, the coordinates in the objective space) and a binary flag indicating whether a solution is part of the current generation of solutions.
- **Edges:** Directed edges are formed from parent to offspring solutions. These edges represent the parent-child relationships that describe how solutions are derived over successive generations. Additionally, in the first generation, self-loops are added to nodes to preserve their existence and relevance despite the absence of ancestors.
- **Edge Features:** Each edge is annotated with the normalized generation index as weight, indicating the moment of transition from parent to offspring. This reflects the relative age of the relationship within the overall search process.

We visualize the shaping of the state space in Figure 2. This formulation provides a comprehensive view of the optimization process, allowing the DRL algorithm to track how solutions evolve and interact over time, thereby offering insight into the convergence process and the solution diversity. It also inherently accounts for the relationships between multiple objectives, providing a holistic view of how solutions navigate trade-offs between conflicting objectives in the search space.

Action Space A **-** The action space consists of adjustments to the evolutionary parameters that govern the search process, e.g., the mutation rates and crossover probabilities. These values are normalized between -1 and 1, defined based on the recommended values from rules of thumb for EA tuning [7]. By adjusting these parameters, STN-MODAC influences how the population evolves over time, guiding the search towards better solutions.

Transition Function T **-** In STN-MODAC, the transition function reflects the effect of evolutionary operations, such as mutation, crossover, and selection operators, on the population and its representation within the Multi-Objective Search Trajectory Network (MO-STN). These dynamics determine how the algorithm navigates between exploration (generating diverse solutions) and exploitation (improving known ones), ultimately shaping the search trajectory. While the state and action spaces are consistent, the reward and transition functions are instance-specific and vary with each problem instance $i \in I$. Consequently, the same action applied in the same state may lead to different outcomes depending on the characteristics of an instance at stake.

Reward Function R **-** In multi-objective optimization, the reward function should actively promote convergence towards the optimal Pareto front. However, progress towards this front becomes increasingly challenging as the optimization advances. Early iterations often yield substantial improvements with minimal effort, but later stages require more sophisticated exploration to make even small gains. To address this, our reward function specifically incentivizes progress in these more demanding phases of the search.

At each iteration t, the reward is calculated based on changes in hypervolume, a common indicator of Pareto front quality. If the current hypervolume HV_{current} surpasses the highest previously recorded value HV_{best}, we calculate the percentage improvements Δ_{current} and Δ_{best}, and then calculate the reward as the difference between the squared improvements. This formulation incentivizes larger improvements, particularly during the later stages of optimization. As a result, it drives the algorithm to pursue more impact full advancements, even when progress becomes harder to achieve. The percentage improvements are calculated as follows:

$$\Delta_{\text{current}} = \left(\frac{HV_{\text{current}} - HV_{\text{initial}}}{HV_{\text{ideal}} - HV_{\text{initial}}} \right) \times 100,$$
$$\Delta_{\text{best}} = \left(\frac{HV_{\text{best}} - HV_{\text{initial}}}{HV_{\text{ideal}} - HV_{\text{initial}}} \right) \times 100.$$

with the reward r_t calculated as the difference between the squared improvements:

$$r_t = \begin{cases} \Delta^2_{\text{current}} - \Delta^2_{\text{best}} & \text{if } HV_{\text{current}} > HV_{\text{best}}, \\ 0 & \text{otherwise.} \end{cases}$$

We compute hypervolumes using a nadir point derived from the worst objective values in the initial population. The reference hypervolume HV_{ideal} is established using this nadir point and an approximate ideal point, obtained by executing the base evolutionary algorithm once with an extended budget (e.g., twice the standard). This computation is performed only once per problem instance, prior to training or evaluation. Consequently, it does not impact the evaluation time during training or testing. Importantly, the proposed reward function is instance-agnostic, enabling its use across a wide range of problem instances with differing sizes and complexities.

3.3 Graph-based Policy and Training Algorithm

We use the Proximal Policy Optimization (PPO) algorithm [24] to train the parameterized policy. PPO is a widely used and highly effective policy gradient algorithm that uses a probability ratio between policies to maximize the improvement of the current policy without the risk of performance collapse. The agent utilizes a neural network that processes the STN-based state representation through a Temporal Graph Convolutional Network (T-GCN). This network combines two layers of spatial Graph Convolutional Networks (GCNs) [11] with a Gated Recurrent Unit (GRU) to capture both the structural and temporal dynamics of the optimization process. The GCN layers extract node embeddings from the current state of the Search Trajectory Network (STN), capturing structural information such as convergence fronts, solution clustering, and trade-offs in the objective space. These embeddings are then passed through a GRU that aggregates information across a fixed-length sequence of previous states, allowing the model to learn how the search process evolves and enabling the agent to anticipate search dynamics, rather than only reacting to the current state. After processing through the T-GCN, a global mean pooling operation is applied to the embeddings of the output nodes to produce a compact representation of the entire graph sequence. This pooled embedding is then concatenated with an auxiliary feature vector that contains the search budget. The resulting enriched feature vector is passed through a fully connected linear layer to predict the mean values of the action distributions.

4 Experiments

4.1 FJSP Instances

We assess the effectiveness of STN-MODAC on a diverse set of flexible job shop scheduling problems (FJSP), including both synthetic benchmarks and complex, real-world-inspired scenarios.

FJSP is a classic combinatorial optimization problem involving the assignment of n independent jobs $J = \{J_1, J_2, \ldots, J_n\}$ to m machines $M = \{M_1, M_2, \ldots, M_m\}$. Each job J_i consists of a sequence of operations $O_{i,j}$ that must be executed in a predefined order, such that the operation $O_{i,j+1}$ can only start after $O_{i,j}$ is complete. Each operation can be processed by a subset of machines, each with a known processing time $t_{i,j,k}$ for machine M_k. Based on the literature [29], we focus on five commonly used minimization objectives:

- Makespan: The total time required to complete all jobs, represented as $C_{\max} = \max_{i=1,\ldots,n} C_{i,j}$.
- Balanced Workload: The disparity in the distribution of the workload between machines, represented as $W_{\text{bal}} = W_{\max} - W_{\min}$, where $W_{\min} = \min_{k=1,\ldots,m} \sum_{(i,j)\in O_k} t_{i,j,k}$.
- Average flowtime: The average duration of time jobs take from start to completion $F_{\text{avg}} = \frac{1}{n}\sum_{i=1}^{n}(C_{i,\text{last}} - S_{i,\text{first}})$.
- Total Workload: The cumulative sum of processing times for all jobs, defined as $W_{\text{total}} = \sum_{k=1}^{m} \sum_{(i,j)\in O_k} t_{i,j,k}$.
- Maximum flowtime: Denoting the longest time any job spends in the system from start to completion, defined as $F_{\max} = \max_{i=1,\ldots,n}(C_{i,\text{last}} - S_{i,\text{first}})$.

We refer to the FJSP variants as Bi-FJSP, Tri-FJSP, and Penta-FJSP, corresponding to solving the first two, first three, and all five objectives, respectively.

Additionally, we consider a more complex problem variant that extends the Bi-, Tri-, and Penta-FJSP with assembly constraints and sequence-dependent setup times. Assembly constraints introduce additional precedence relations between different jobs. Specifically, if job J_p must be assembled before job J_q, we require $C_{p,\text{last}} \leq S_{q,\text{first}}$, where $C_{p,\text{last}}$ is the completion time of the last operation of job J_p, and $S_{q,\text{first}}$ is the start time of the first operation of job J_q. Sequence-dependent setup times introduce additional times required to switch between operations on the same machine. If operation $O_{i,j}$ is immediately followed by operation $O_{i',j'}$ on machine M_k, the start time constraint is $S_{i',j'} \geq C_{i,j} + s_{(i,j),(i',j'),k}$, where $s_{(i,j),(i',j'),k}$ denotes the setup time on machine M_k between operations $O_{i,j}$ and $O_{i',j'}$. We refer to these problems as the Bi-, Tri-, and Penta-YFJS-SDST scheduling problems, following the problem formulation introduced by [4], which has been further extended to include sequence-dependent setup times.

4.2 Baselines

To evaluate the performance of STN-MODAC, we adopt NSGA-II as the base algorithm, following the operator formulations of [20]. The operator parameters are configured using common rules of thumb: a crossover rate of 0.7 and a mutation rate of 0.02 [7]. We further compare the proposed method against five algorithm configuration approaches for tuning the parameters of NSGA-II: three widely used static configuration methods, SMAC3 [13], MO-SMAC [23], and irace [14], as well as two recent reinforcement learning-based dynamic configuration methods, MADAC [32] and GS-MODAC [22].

SMAC3 is a hyperparameter tuning method that integrates Bayesian optimization with random forest regression. For tuning, we used generated test instances for each problem configuration. Bayesian optimization selects parameter configurations from a predefined range and evaluates them on tuning instances over $10,000$ runs of NSGA-II. This process takes between 24 hours for the FJSP variants with 10 jobs and 5 machines. MO-SMAC extends this approach for multi-objective optimization problems, approximating the Pareto-optimal set of configurations.

irace (Iterated Racing) follows racing-based procedure that iteratively eliminates poorly performing configurations and introduces new ones based on performance over a training set of instances. irace is allocated the same computational budget as SMAC3 and completes in about 16 hours for the FJSP problem variants.

MADAC is a multi-agent reinforcement learning approach that employs value-decomposition networks (VDN) [28] to determine optimal settings for different categories of parameters. It integrates information from the specific problem instance, the current state of the optimization process, and the evolving population of solutions.

Table 1: Comparison of performance across various methods in solving 100 instances of Bi-, Tri-, and Penta-FJSP, each configured with 10 jobs and 5 machines. Each instance is solved 10 times, and the results are evaluated based on the hypervolume (mean), the best hypervolume found (max), and the standard deviation (std).

	Metric	NSGA-II	irace	SMAC3	MO-SMAC	MADAC	GS-MODAC	STN-MODAC **(ours)**
Bi-FJSP-10j5m	mean	3.82×10^{4}	3.90×10^{4}	3.89×10^{4}	3.86×10^{4}	3.69×10^{4}	3.92×10^{4}	$\mathbf{3.94\times10^{4}}$
	max	4.11×10^{4}	4.11×10^{4}	4.13×10^{4}	4.12×10^{4}	3.98×10^{4}	4.15×10^{4}	$\mathbf{4.17\times10^{4}}$
	std	2.29×10^{3}	1.95×10^{3}	2.19×10^{3}	2.56×10^{3}	4.47×10^{3}	1.97×10^{3}	2.22×10^{3}
Tri-FJSP-10j5m	mean	5.53×10^{6}	5.47×10^{6}	5.65×10^{6}	5.63×10^{6}	5.39×10^{6}	$\mathbf{5.70\times10^{6}}$	5.68×10^{6}
	max	5.95×10^{6}	5.82×10^{6}	6.05×10^{6}	6.02×10^{6}	5.87×10^{6}	6.09×10^{6}	$\mathbf{6.11\times10^{6}}$
	std	3.09×10^{5}	2.65×10^{5}	2.91×10^{5}	3.40×10^{5}	5.97×10^{5}	2.99×10^{5}	3.58×10^{5}
Penta-FJSP-10j5m	mean	3.96×10^{11}	4.03×10^{11}	3.97×10^{11}	3.88×10^{11}	3.91×10^{11}	4.16×10^{11}	$\mathbf{4.17\times10^{11}}$
	max	4.31×10^{11}	4.38×10^{11}	4.29×10^{11}	4.29×10^{11}	4.29×10^{11}	4.52×10^{11}	$\mathbf{4.53\times10^{11}}$
	std	2.42×10^{10}	2.35×10^{10}	2.24×10^{10}	2.20×10^{10}	4.36×10^{10}	2.40×10^{10}	2.62×10^{10}

As it is designed to select discrete actions, we discretize the parameter space of NSGA-II into 10 actions, setting crossover rates between 0.6 and 1.0 and mutation rates between 0 and 0.1, following the established evolutionary algorithm parameter guidelines [7].

GS-MODAC is a dynamic algorithm configuration method that combines Graph Neural Networks (GNNs) and Deep Reinforcement Learning (DRL) to adaptively control the parameters of multi-objective evoluti5onary algorithms. It formulates the configuration process as a contextual Markov Decision Process (MDP), where states are represented by graphs constructed from the normalized objective values of the current solution population. Like MADAC, GS-MODAC operates within the same discretized parameter ranges for crossover and mutation rates.

4.3 Instances, Training and Testing

Regarding the FJSP problem variants, we generate training and test instances for two FJSP problem sizes, for instances with 10 jobs and 5 machines (10j5m), and for instances with 50 jobs and 5 machines (50j5m). For each instance size, we generate 200 instances, 100 instances for training, and 100 for testing. Each instance is configured with jobs with a varying number of operations per job, ranging between 4 and 8, with processing times between 2 and 20 time units. The same instance sets are used for experiments with the Bi- Tri- and Penta problem variants. The instances of YFJS-SDST problems are generated according to the work of [4], for which we have generated sequence-dependent setup times that last between 5 and 25 timesteps. For this variant, we generated 20 training instances and used the 20 publicly available instances from the original work for testing.

STN-MODAC was trained for each problem configuration using the generated set of training instances. The underlying optimization was guided by NSGA-II, which was controlled through a continuous two-dimensional action space. These actions adjusted the crossover and mutation rates, constrained to the intervals $\langle 0.6, 1.0\rangle$ and $\langle 0.0, 0.1\rangle$, respectively. Training was performed over 1,000,000 steps, configured by 2000 epochs with 500 steps per epoch. Each NSGA-II evolutionary run is configured with 50 generations and a population size of 50. The training pipeline was executed on an Intel(R) Core(TM) i7-6920HQ CPU @ 2.90GHz with 8 GB of RAM, using five parallel environments to accelerate learning. Training each of the Bi-, Tri-, and Penta-FJSP problem variants (with 10 jobs and 5 machines) required approximately 40 hours. The DRL algorithm followed the parameterization proposed by [24], and each layer of the network architecture consisted of 64 nodes. GS-MODAC was trained according to the setup of [22] for 1.000,000 steps, taking approximately 15 hours of training on the 10j5m-FJSP problem variants. The MADAC baseline model is trained following the setup of [32], which takes 30 hours for the FJSP-based variants.

The trained models are applied to fine-tune the parameters of NSGA-II to solve previously unseen problem instances. Each experiment is conducted by running both algorithms 10 times on a set of 100 test instances to facilitate comparison. Performance is assessed using three key metrics: average hypervolume (mean), best hypervolume (max), and standard deviation (std), all of which are derived by averaging results across all test instances for each problem. To ensure consistency in evaluation, hypervolumes are calculated using a fixed set of reference points for each test instance. The paper emphasizes the best performing results in bold (highest mean and maximum hypervolumes), with underlining indicating statistically significant improvements over all other methods, as verified by the Wilcoxon rank sum test ($p < 0.05$).

4.4 Results

In this section, we present the results of our experiments to evaluate the performance of various optimization methods for solving multi-objective scheduling problems. We first compare the performance of the proposed method against the baselines and then examine the scalability of the STN-MODAC agent when trained on smaller instances and applied to larger ones. Subsequently, we assess the ability to be transferred to more constrained problem variants that were not observed in the training process.

RQ1: How does the performance of STN-MODAC compare to existing baseline methods? Table 1 presents the performances of the various methods in solving 100 instances of Bi-, Tri- and Penta-FJSP with 10 jobs and 5 machines evaluated on three key metrics: average hypervolume (mean), best hypervolume found (max), and standard deviation (std). The results highlight that across all problem configurations, the proposed STN-MODAC method consistently demonstrates strong and reliable performance. In terms of average and maximum hypervolumes, STN-MODAC either matches or outperforms all baselines. Specifically, it achieves the highest average hypervolume in the Bi-FJSP and Penta-FJSP settings, and the best maximum hypervolumes in the evaluated problems.

RQ2: Can STN-MODAC trained on small-scale scheduling instances generalize effectively to larger problem sizes? Table 2 presents the results of a transferability study conducted to evaluate the scalability of the STN-MODAC agent. The agent was trained on relatively small instances (10 jobs, 5 machines) and then applied to larger instances with 50 jobs and 5 machines. Across all problem configurations, STN-MODAC achieved the highest average and best hypervolume values, outperforming all competing methods. This demonstrates the strong scalability capability of STN-MODAC,

Table 2: Evaluation of the transferability of the STN-MODAC agent trained on instances with 10 jobs and 5 machines to solve instances configured with 50 jobs and 5 machines. All baselines were trained/tuned exclusively on the small instances and then directly applied without retraining or fine-tuning to the larger problem sizes. The best-performing results are shown in bold (indicating the highest mean and maximum hypervolumes), with underlining denoting statistically significance improvements over all other methods, as verified by the Wilcoxon rank sum test ($p < 0.05$).

	Metric	NSGA-II	irace	SMAC3	MADAC	GS-MODAC	STN-MODAC **(ours)**
Bi-FJSP-50j5m	mean	1.92×10^{5}	2.00×10^{5}	2.01×10^{5}	1.94×10^{5}	**2.05×10^{5}**	**2.05×10^{5}**
	max	2.05×10^{5}	2.11×10^{5}	2.13×10^{5}	2.12×10^{5}	2.15×10^{5}	**2.16×10^{5}**
	std	1.19×10^{4}	1.05×10^{4}	1.17×10^{4}	1.99×10^{4}	1.13×10^{4}	1.26×10^{4}
Tri-FJSP-50j5m	mean	6.78×10^{7}	7.21×10^{7}	7.35×10^{7}	7.24×10^{7}	7.77×10^{7}	**7.82×10^{7}**
	max	7.30×10^{7}	7.68×10^{7}	7.84×10^{7}	7.78×10^{7}	8.24×10^{7}	**$\underline{8.38\times10^{7}}$**
	std	3.94×10^{6}	3.92×10^{6}	4.13×10^{6}	3.32×10^{6}	4.28×10^{6}	5.04×10^{6}
Penta-FJSP-50j5m	mean	5.08×10^{13}	5.66×10^{13}	5.68×10^{13}	5.57×10^{13}	6.53×10^{13}	**$\underline{6.63\times10^{13}}$**
	max	5.49×10^{13}	6.16×10^{13}	6.15×10^{13}	6.01×10^{13}	7.13×10^{13}	**$\underline{7.20\times10^{13}}$**
	std	2.85×10^{12}	3.28×10^{12}	3.29×10^{12}	2.55×10^{12}	4.41×10^{12}	4.37×10^{12}

Table 3: Evaluation of the transferability of the STN-MODAC agent trained on instances with 10 jobs and 5 machines to solve instances of a different constrainted problem variant, with assembly constraints and sequence dependent setup times [4]. All baselines were trained/tuned exclusively on the FJSP problem variant and then directly applied without retraining or fine-tuning.

	Metric	NSGA-II	irace	SMAC3	MADAC	GS-MODAC	STN-MODAC **(ours)**
Bi-YFJS-SDST	mean	3.75×10^{6}	4.02×10^{6}	4.03×10^{6}	3.79×10^{6}	**4.20×10^{6}**	**4.20×10^{6}**
	max	4.02×10^{6}	4.24×10^{6}	4.26×10^{6}	4.40×10^{6}	4.43×10^{6}	**4.44×10^{6}**
	std	2.48×10^{5}	1.78×10^{5}	1.79×10^{5}	3.62×10^{4}	2.10×10^{5}	$2.81\text{x}10^{5}$
Tri-YFJS-SDST	mean	3.86×10^{9}	3.87×10^{9}	3.90×10^{9}	**4.31×10^{9}**	4.17×10^{9}	4.27×10^{9}
	max	4.15×10^{9}	4.13×10^{9}	4.17×10^{9}	4.46×10^{9}	4.55×10^{9}	**$\underline{4.67\times10^{9}}$**
	std	2.14×10^{8}	1.98×10^{8}	2.04×10^{8}	9.63×10^{7}	3.48×10^{8}	3.66×10^{8}
Penta-YFJS-SDST	mean	2.37×10^{17}	2.38×10^{17}	2.43×10^{17}	2.58×10^{17}	2.65×10^{17}	**$\underline{2.66\times10^{17}}$**
	max	2.60×10^{17}	2.60×10^{17}	2.65×10^{17}	2.67×10^{17}	2.95×10^{17}	**$\underline{3.07\times10^{17}}$**
	std	1.99×10^{16}	1.98×10^{16}	1.90×10^{16}	5.08×10^{15}	2.46×10^{16}	2.98×10^{16}

which maintains high performance even when applied to problem sizes that are beyond those seen during training. In particular, while GS-MODAC performed competitively when trained and tested on the same instance sizes (as shown in Table 1), its performance degraded noticeably in this transfer scenario, highlighting the advantage of STN-MODAC's temporally-aware, graph-based policy representation in adapting to larger problem instances.

RQ3: Can STN-MODAC trained on standard FJSP problems be transferred to more complex, real-world problem variants? To further assess the applicability of trained STN-MODAC agents, we evaluated their performance on a real-world-inspired variant of the scheduling problem, namely the Bi-, Tri-, and Penta-YFJS-SDST, which introduces sequence-dependent setup times (SDST) and assembly constraints to the FJSP problem. We assess how the different methods trained and tuned FJSP problem with 10j5m instances perform when deployed on this more complex problem variant, and present the results in Table 3.

The table shows that the STN-MODAC agent shows strong performance across all configurations of YFJS-SDST problems. For the Bi-YFJS-SDST instance with 50 jobs and 5 machines, STN-MODAC outperforms all baselines in terms of both mean and maximum hypervolume. Specifically, the mean hypervolume achieved by STN-MODAC is slightly better than the best-performing baseline method (GS-MODAC). Similarly, for the Tri- and Penta-YFJS-SDST variants, STN-MODAC shows consistent improvement in both average and maximum hypervolume metrics, being effective in tackling problem configurations that include additional constraints not observed during training. These results demonstrate the robustness and adaptability of the STN-MODAC agent, not only for synthetic problem instances, but also to solve more constrained scheduling problems that were not observed during training.

5 Conclusion

This paper introduces the *Search Trajectory Network based Multi-Objective Dynamic Algorithm Configuration (STN-MODAC)* method, which leverages search trajectories, Graph Neural Networks (GNNs), and Deep Reinforcement Learning (DRL) to dynamically configure Evolutionary Algorithms for multi-objective combinatorial optimization problems. The proposed approach models the evolving population dynamics of a multi-objective evolutionary algorithm within a Multi-Objective Search Trajectory Network, a temporal graph structure that encodes the relationships between solutions across generations. This network is then processed using a Temporal Graph Convolutional Network (T-GCN) to extract both spatial and temporal features that reflect the current search status. These graph-based features are fed into a DRL agent, which learns a policy to dynamically reconfigure the algorithm in real time, leading to improved convergence. Experimental results demonstrate that STN-MODAC not only improves convergence, but also significantly outperforms existing baselines in terms of transferability to larger problem instances and more complex problem variants, highlighting its strong generalization capability.

Acknowledgements

This work is supported by the Luxembourg National Research Fund (FNR) (15706426).

References

[1] A. Aleti and I. Moser. A systematic literature review of adaptive parameter control methods for evolutionary algorithms. *ACM Computing Surveys (CSUR)*, 49(3):1–35, 2016.

[2] L. Begnardi, B. von Meijenfeldt, Y. Zhang, W. van Jaarsveld, and H. Baier. Algorithm configuration in sequential decision-making. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 86–102. Springer, 2025.

[3] A. Biedenkapp, H. F. Bozkurt, T. Eimer, F. Hutter, and M. Lindauer. Dynamic algorithm configuration: Foundation of a new meta-algorithmic framework. In *ECAI 2020*, pages 427–434. IOS Press, 2020.

[4] E. G. Birgin, P. Feofiloff, C. G. Fernandes, E. L. De Melo, M. T. Oshiro, and D. P. Ronconi. A milp model for an extended version of the flexible job shop problem. *Optimization Letters*, 8:1417–1431, 2014.

[5] P. Bosman and A. P. Engelbrecht. Diversity rate of change measurement for particle swarm optimisers. In *International Conference on Swarm Intelligence*, pages 86–97. Springer, 2014.

[6] R. Chen, B. Yang, S. Li, and S. Wang. A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering*, 149:106778, 2020.

[7] C. A. C. Coello, G. B. Lamont, D. A. Van Veldhuizen, et al. *Evolutionary algorithms for solving multi-objective problems*, volume 5. Springer, 2007.

[8] J. E. Fieldsend, T. Chugh, R. Allmendinger, and K. Miettinen. A feature rich distance-based many-objective visualisable test problem generator. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 541–549, 2019.

[9] Y. Huang, W. Li, F. Tian, and X. Meng. A fitness landscape ruggedness multiobjective differential evolution algorithm with a reinforcement learning strategy. *Applied Soft Computing*, 96:106693, 2020.

[10] G. Karafotias, M. Hoogendoorn, and Á. E. Eiben. Parameter control in evolutionary algorithms: Trends and challenges. *IEEE Transactions on Evolutionary Computation*, 19(2):167–187, 2014.

[11] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[12] Y. Lavinas, C. Aranha, and G. Ochoa. Search trajectories networks of multiobjective evolutionary algorithms. In *International Conference on the Applications of Evolutionary Computation (Part of EvoStar)*, pages 223–238. Springer, 2022.

[13] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter. Smac3: A versatile bayesian optimization package for hyperparameter optimization. *J. Mach. Learn. Res.*, 23(54):1–9, 2022.

[14] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, M. Birattari, and T. Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

[15] Z. Ma, H. Guo, Y.-J. Gong, J. Zhang, and K. C. Tan. Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization. *arXiv preprint arXiv:2411.00625*, 2024.

[16] Z. Michalewicz, D. B. Fogel, and T. Bèack. *Evolutionary Computation. Vol. 2, Advanced Algorithms and Operators*. Taylor & Francis, 2000.

[17] G. Ochoa, K. M. Malan, and C. Blum. Search trajectory networks: A tool for analysing and visualising the behaviour of metaheuristics. *Applied Soft Computing*, 109:107492, 2021.

[18] J. Quevedo, M. Abdelatti, F. Imani, and M. Sodhi. Using reinforcement learning for tuning genetic algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 1503–1507, 2021.

[19] R. Reijnen, Y. Zhang, Z. Bukhsh, and M. Guzek. Deep reinforcement learning for adaptive parameter control in differential evolution for multi-objective optimization. In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 804–811. IEEE, 2022.

[20] R. Reijnen, K. van Straaten, Z. Bukhsh, and Y. Zhang. Job shop scheduling benchmark: Environments and instances for learning and non-learning methods. *arXiv preprint arXiv:2308.12794*, 2023.

[21] R. Reijnen, Y. Zhang, H. C. Lau, and Z. Bukhsh. Online control of adaptive large neighborhood search using deep reinforcement learning. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 475–483, 2024.

[22] R. Reijnen, Y. Wu, Z. Bukhsh, and Y. Zhang. Graph-supported dynamic algorithm configuration for multi-objective combinatorial optimization. In *International Conference on Machine Learning*. PMLR, 2025.

[23] J. G. Rook, C. Benjamins, J. Bossek, H. Trautmann, H. H. Hoos, and M. Lindauer. Mo-smac: Multiobjective sequential model-based algorithm configuration. *Evolutionary Computation*, pages 1–24, 2025.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[25] M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov. Deep reinforcement learning based parameter control in differential evolution. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 709–717, 2019.

[26] G. Sun, G. Xu, and N. Jiang. A simple differential evolution with time-varying strategy for continuous optimization. *Soft Computing*, 24(4): 2727–2747, 2020.

[27] J. Sun, X. Liu, T. Bäck, and Z. Xu. Learning adaptive differential evolution algorithm from optimization experiences by policy gradient. *IEEE Transactions on Evolutionary Computation*, 25(4):666–680, 2021.

[28] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296*, 2017.

[29] K. Tamssaouet, S. Dauzère-Pérès, S. Knopp, A. Bitar, and C. Yugma. Multiobjective optimization for complex flexible job-shop scheduling problems. *European Journal of Operational Research*, 296(1):87–100, 2022.

[30] Y. Tian, X. Li, H. Ma, X. Zhang, K. C. Tan, and Y. Jin. Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 7(4):1051–1064, 2022.

[31] T. Tušar and B. Filipič. Visualization of pareto front approximations in evolutionary multiobjective optimization: A critical review and the prosection method. *IEEE Transactions on Evolutionary Computation*, 19(2):225–245, 2014.

[32] K. Xue, J. Xu, L. Yuan, M. Li, C. Qian, Z. Zhang, and Y. Yu. Multi-agent dynamic algorithm configuration. *Advances in Neural Information Processing Systems*, 35:20147–20161, 2022.

[33] X. Yang, R. Wang, K. Li, and H. Ishibuchi. Meta-black-box optimization for evolutionary algorithms: Review and perspective. *Swarm and Evolutionary Computation*, 93:101838, 2025.

[34] E. Zitzler and L. Thiele. Multiobjective optimization using evolutionary algorithms—a comparative case study. In *International conference on parallel problem solving from nature*, pages 292–301. Springer, 1998.